# Hidden in the Leftovers

Task-Specific Reconstruction Gaps of Sparse Autoencoders
in English and Indian Languages

Prakhar Singhal (2022111025, prakhar.singhal@research.iiit.ac.in)
Raveesh Vyas (2022114002, raveesh.vyas@research.iiit.ac.in)

Mechanistic Interpretability of Language Models (CL3.412)
Course project proposal · Team IpCp-- · October 2026

**Summary.** Sparse autoencoders (SAEs) split a language model's internal activations into interpretable features. They are usually judged by average reconstruction quality, and the widely used Gemma Scope SAEs score 98–99%. An average, however, can hide what is lost on a specific task. Our pilot on gemma-2-2b finds such a loss. On an object-tracking task, the SAE's features carry only 74% of the information the model uses at layer 12. One direction in the SAE's leftover error, estimated on this task, restores about half of the missing part. Directions from other tasks or from ordinary text restore almost nothing. We propose to test how common such task-specific gaps are, whether this one is part of how the model links objects to locations, and whether gaps are larger in Hindi and Hinglish.

## 1 What: research question and hypothesis

**Background.** A language model represents each token (a word or word piece) at each layer by an **activation** $x$. In gemma-2-2b this is a vector of 2304 numbers. A **sparse autoencoder** (SAE) writes it as $x = \hat{x} + \varepsilon$. The **features** $\hat{x}$ are built from a few interpretable components. The **SAE error** $\varepsilon$ is what is left over, often called "dark matter" [4]. A **direction** is a fixed vector in this space. Appendix D defines all terms.

**The gap.** Take two prompts whose answers differ. If the features captured everything the model uses, swapping only the features between the prompts would change the answer as much as swapping the whole activation. The share of that change the features miss is the SAE's **gap** on the task.

**Research question.** *When an SAE's features miss information that a model uses for a task, is the missing part specific to that task, and is more of it missed in Indian languages?*

**Hypothesis.** Gaps come in two kinds. A *concentrated, task-specific* gap is mostly closed by adding back the error along one or two directions estimated on that task. A *diffuse* gap needs many directions. Average reconstruction quality cannot tell them apart. Our pilot (Section 3) found both. We ask three questions:

- **Q1. Generality.** Which kinds of task show a concentrated, task-specific gap? Is the gap tied to one prompt template or to a type of computation?
- **Q2. Mechanism.** In object tracking, is the error direction part of the model's *binding* mechanism, the way it links an object to its location [5]?
- **Q3. Language.** Are the gaps larger in Hindi and Hinglish than in English? Do the same directions close them?

## 2 Why: motivation

SAEs such as Gemma Scope [9] are used to explain, audit and steer models. They are mainly evaluated by **reconstruction fidelity**: the share of next-token performance kept when $\hat{x}$ replaces $x$. For the SAEs we use it is 98–99%. But fidelity is an average over ordinary text. It cannot show what is lost on a specific computation. The SAE error is also known to be sizeable, partly predictable from the input [4], and important inside feature circuits [11].

This matters for how SAEs are used. If what they miss is *generic*, one fix serves every task, such as a wider SAE or a correction fitted on ordinary text. If it is *task-specific*, a feature-based explanation of a task can be incomplete exactly where fidelity does not look. Language raises the same question. The Gemma Scope SAEs were trained mostly on English text [9]. Prior work finds language-specific SAE features [1, 2], but does not measure what the error carries in each language.

**Table 1:** The four pilot tasks, with one real pair each. Prompt B changes one step of prompt A.

| Task | Prompt A (answer) | Change in prompt B (answer) |
|---|---|---|
| Running sum | `Start with 2. Add 3. Add 1. Add 2. Total =` (8) | `Add 1` becomes `Add 3` (10) |
| Object tracking [8] | `The pen is in box B. The coin is in box G. The key is in box C. The coin moves to box A. The pen moves to box G. The coin is in box` (A) | the second move is made by the coin (G) |
| Variable copying | `y = 4. r = 4. x = r. r = 3. x =` (4) | `r = 4` becomes `r = 7` (7) |
| Names (IOI) [14] | `When Lucy and David went to the river, David gave a drink to` (Lucy) | Lucy is the giver (David) |

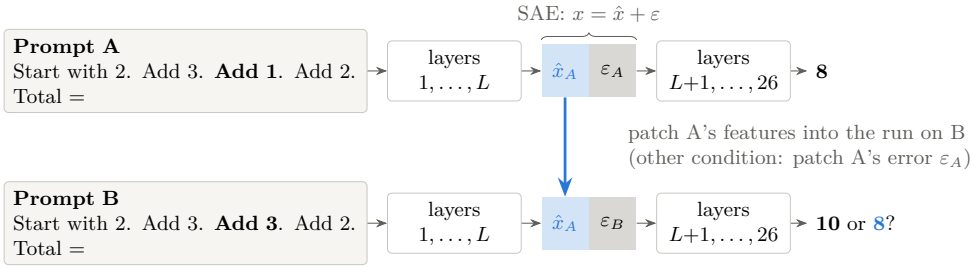



**Figure 1:** Activation patching with an SAE. At layer $L$ the SAE splits the activation into features $\hat{x}$ and error $\varepsilon$. If A's features alone move B's answer from 10 to 8, the features carry the task's information.

## 3 Pilot study: what we found

**Setup.** We use gemma-2-2b, an open 2-billion-parameter model [6], with Gemma Scope SAEs of 16k to 262k features (Appendix A.1). Each task is a set of prompt pairs. The two prompts in a pair differ in one step, so their answers differ (Table 1). We estimate directions on 300 pairs and report every number on 150 held-out pairs. Three tasks require the model to track a changing state; the names task (IOI) is a well-studied control.

**Method: activation patching** [15]. We run the model on prompt B. At one layer, we replace part of its activation ($\hat{x}$ or $\varepsilon$) with the same part from prompt A (Figure 1). The **effect recovered** measures how far this moves the answer from B's towards A's. 0% means no change; 100% means as far as swapping the whole activation. The gap is 100% minus the effect recovered by the features.

**Findings** (full results in Appendix B):

1. **The features carry most of what the model uses.** On the three tracking tasks, the features recover 74–100% of the effect at every layer we tested. The error alone recovers little. Most of what it does recover also appears when we simply replace it with random noise.
2. **One gap is concentrated and task-specific.** For object tracking at layer 12, the features recover 74%, so the gap is 26%. Adding back A's error along one direction from this task raises this to 88%. That closes about half of the gap (Figure 2). Directions from other tasks, ordinary text or random choice close at most 2%. The running sum at layer 18 shows the same pattern on a smaller gap (13%).
3. **Other gaps are diffuse.** The names task loses 24–36% at layers 18–20. No small set of directions explains this loss: closing half of it takes 16 to 32 directions.
4. **Wider SAEs shrink the gap but do not remove it.** From 16k to 262k features, the object-tracking gap falls from 26% to 15%. One error direction still closes more than half of what remains.

Three measurement choices can reverse these conclusions if made carelessly; Appendix B.4 shows them.

**Limitations.** The pilot uses one model, one SAE family and synthetic tasks. Width was varied at one layer only, and wider SAEs in this family also use more features per token. Patching along a few directions can create effects the model does not normally use [10]. Our random and cross-task controls address this only in part.

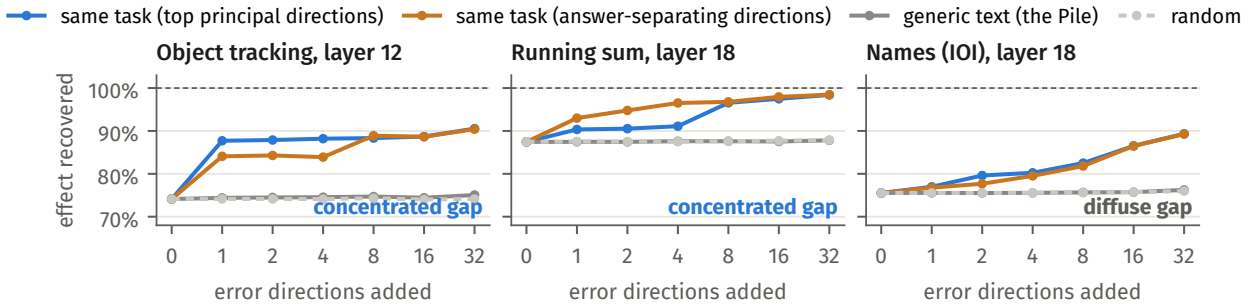



**Figure 2:** Concentrated and diffuse gaps: effect recovered when A's features are patched together with A's error along $N$ directions. Same-task directions close much of the gap with one or two directions (left, middle). Generic-text and random directions close nothing. The names gap closes only slowly (right).

## 4 How: plan and success criteria

**Q1, generality.** We group tasks by the computation they need: *entity binding* (objects in boxes; people and their cities, e.g. "Asha lives in Pune. Ravi lives in Delhi. Asha lives in"), *counting* (longer running sums), *reference following* (variable copying) and the names control. For each task, layer and SAE width we measure the gap, the number of directions that close half of it, and a **specificity score**. This score is the share closed by same-task directions minus the largest share from any other source. If directions transfer between the two binding tasks, the gap belongs to the computation, not to one template. Comparing SAEs with equal numbers of active features separates width from sparsity.

**Q2, mechanism.** Feng and Steinhardt [5] found that LLaMA and Pythia models tag each entity and its attribute with a shared "binding ID" vector. We will estimate these vectors in gemma-2-2b and compare them with the object-tracking error direction. We will then shift activations along that direction. If it encodes binding, the answer should move to the other box.

**Q3, language.** We will run the same tasks in Hindi (Devanagari), Hinglish (Latin script mixed with English words) and Hindi in Latin script without English words. We add Tamil or Bengali if the model can do the tasks in them, and French as a non-Indian control. Example, the running sum in Hindi: शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल = → 8. Appendix C gives the design.

**Table 2:** Success criteria, fixed in advance (95% confidence intervals; otherwise inconclusive).

| Question | Supported if | Not supported if |
|---|---|---|
| Q1 | gap $\geq$ 10%, and same-task directions close $\geq$ 30 points more than any other source; transfer between binding tasks $\geq$ half of own | on $\geq$ 2 new tasks with a gap, other sources close as much as same-task directions |
| Q2 | shifting along the direction moves $\geq$ 50% of pairs to the other box; a random direction $\leq$ 5% | the shift is no more effective than a random direction |
| Q3 | Hindi gap larger than English, with non-overlapping intervals (further criteria in Table C.1) | Hindi gap not larger |

**Deliverables.** (1) A map of gap size, type and specificity across tasks, layers, widths and languages. (2) The patching code, with the three measurement pitfalls handled. (3) A tested answer to the binding hypothesis. **Resources.** One 8 GB GPU and the pilot pipeline, which already runs end to end. Risks and fallbacks are in Appendix C.

## References

[1] J. Brinkmann et al. Large language models share representations of latent grammatical concepts across typologically diverse languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6131–6150, Apr. 2025. Association for Computational Linguistics.

[2] B. Deng et al. Unveiling language-specific features in large language models via sparse autoencoders. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4563–4608, July 2025. Association for Computational Linguistics.

[3] S. Doddapaneni et al. Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *Proceedings of the 61st Annual Meeting of the Association*

*for Computational Linguistics (Volume 1: Long Papers)*, pages 12402–12426, July 2023. Association for Computational Linguistics.

[4] J. Engels, L. R. Smith, and M. Tegmark. Decomposing the dark matter of sparse autoencoders. *Transactions on Machine Learning Research*, 2025.

[5] J. Feng and J. Steinhardt. How do language models bind entities in context? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[6] Gemma Team et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[7] J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Nov. 2019. Association for Computational Linguistics.

[8] N. Kim and S. Schuster. Entity tracking in language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3835–3855, July 2023. Association for Computational Linguistics.

[9] T. Lieberum et al. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Nov. 2024. Association for Computational Linguistics.

[10] A. Makelov et al. Is this the subspace you are looking for? An interpretability illusion for subspace activation patching. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[11] S. Marks et al. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[12] G. Penedo et al. FineWeb2: One pipeline to scale them all — adapting pre-training data processing to every language. In *Second Conference on Language Modeling (COLM)*, 2025.

[13] S. Rajamanoharan et al. Jumping ahead: Improving reconstruction fidelity with JumpReLU sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.

[14] K. R. Wang et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[15] F. Zhang and N. Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

# Appendices

A Pilot methods: model, SAEs, tasks, patching, repair
B Pilot results: features vs error, repair, specificity, width, pitfalls, robustness
C Design of the planned study
D Glossary

## A Pilot methods

### A.1 Model, SAEs and tasks

**Table A.1:** Model and SAEs.

| Item | Value |
|---|---|
| Model | gemma-2-2b: 26 layers, 2304 numbers per token, 16-bit precision |
| SAEs | Gemma Scope residual-stream SAEs with JumpReLU activation [9, 13]; widths 16k, 32k, 65k, 131k and 262k |
| Fidelity (16k) | next-token performance kept on generic text (the Pile): 99.1% at layer 10, 98.7% at 12, 98.5% at 14, 98.0% at 18 |
| Error size (16k) | $\|\varepsilon\|/\|x\|$ on the task prompts: 0.08–0.18, largest at layers 8–16 |
| Compute | one 8 GB laptop GPU |

Both prompts in a pair have the same number of tokens, and each answer is a single token. The same worked examples are placed in front of both prompts. We keep only pairs the model handles: its top answer must be correct, and it must clearly prefer A's answer on prompt A. For object tracking, the hardest task, the top answer must be correct on prompt A and the model's preference must shift clearly between the two prompts.

**Table A.2:** Task settings. "Kept" is the share of generated pairs the model handles. Chance is one over the number of possible answers. Every task has 300 training and 150 test pairs.

| Task | Difficulty | Worked examples | Kept | Chance |
|---|---|---|---|---|
| Running sum | 3 steps | 8 | 62% | 25% |
| Object tracking | 3 objects, 2 moves | 8 | 27% | 18% |
| Variable copying | 1 copy step | 2 | 40% | 11% |
| Names (IOI) | – | 2 | 100% | 11% |

### A.2 Activation patching

**Effect recovered.** A *logit* is the model's raw score for a possible next token. Let $D$ be the logit of A's answer minus the logit of B's answer. Then the effect recovered is $(D_{\text{patched}} - D_B)/(D_A - D_B)$, averaged over test pairs. It is 0 for no change and 1 for swapping the whole activation; it can exceed 1. Intervals are 95% paired bootstrap intervals.

**Conditions.** The subscript names the prompt each part comes from. *Features only:* $\hat{x}_A + \varepsilon_B$. *Error only:* $\hat{x}_B + \varepsilon_A$. *Noise control:* $\hat{x}_B + \eta$, where $\eta$ is random noise of the same size as $\varepsilon$. It erases B's error without supplying A's.

**Positions.** We patch every token from the first one where A and B differ (Figure A.1). Patching only the last token gives misleading results (Appendix B.4).

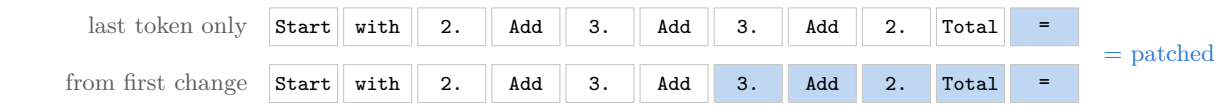



**Figure A.1:** Patch positions for a running-sum pair. Earlier tokens are identical in A and B. Patching from the first changed token covers everything that differs; the last token alone may not yet hold the state.

### A.3 Repair with error directions

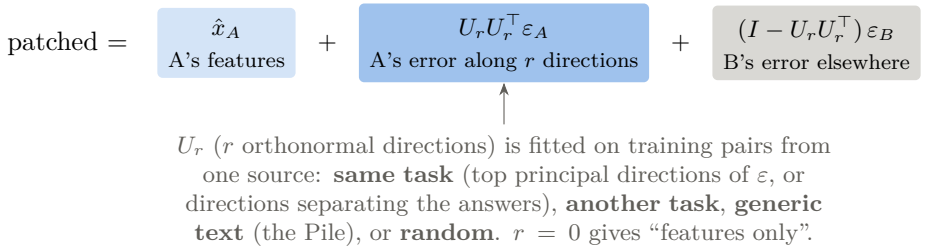



**Figure A.2:** Repair. The run on B receives A's features, A's error along $r$ chosen directions, and B's own error in all other directions.

Directions are estimated on the training pairs only. We use two same-task estimates. *Top principal directions* are where the task's errors vary most. *Answer-separating directions* best separate the average errors for each answer. We compare them with three other sources: the same estimate on another task, on generic text (the Pile), and random directions. We test 0 to 32 directions. The **share of the gap closed** is (repair − features only) / gap.

## B Pilot results

### B.1 Features versus error, by layer

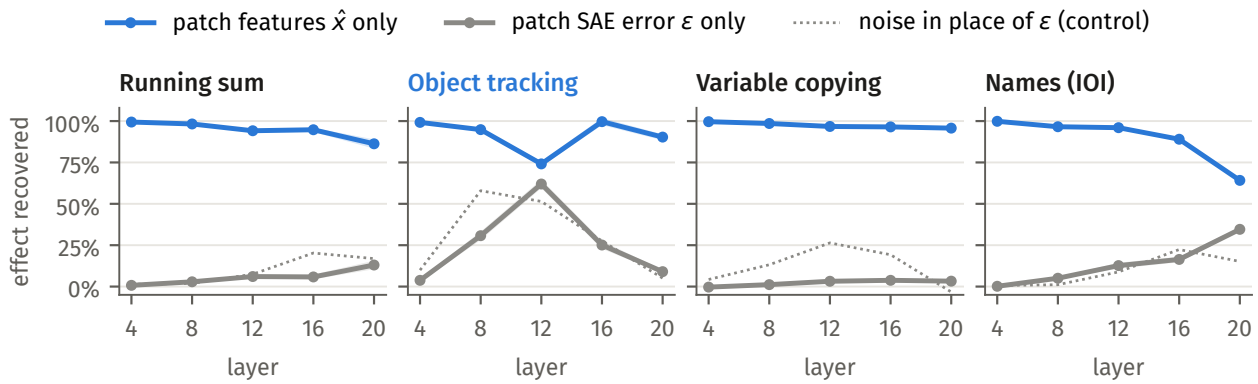



**Figure B.1:** The features carry most of the effect at every layer; the error alone carries little. Shaded bands are 95% intervals. For object tracking, the error curve at layers 8–16 follows the noise control.

**Table B.1:** Effect recovered (%) by patching the features / the error, per layer.

| Task | Layer 4 | Layer 8 | Layer 12 | Layer 16 | Layer 20 |
|---|---|---|---|---|---|
| Running sum | 99 / 1 | 98 / 3 | 94 / 6 | 95 / 6 | 86 / 13 |
| Object tracking | 99 / 4 | 95 / 31 | 74 / 62 | 100 / 25 | 90 / 9 |
| Variable copying | 100 / 0 | 99 / 1 | 97 / 3 | 96 / 4 | 96 / 3 |
| Names (IOI) | 100 / 0 | 97 / 5 | 96 / 13 | 89 / 16 | 64 / 35 |

**Controls at layer 12.** Replacing B's error with random noise recovers 7, 51, 26 and 9% for the four tasks (in table order). So for object tracking, most of the error's 62% comes from erasing B's error, not from supplying A's. The error's effect also does not depend on the answer. For object tracking, an error borrowed from another pair works about as well whether that pair's answer matches A's or not (28 vs 26%, last token only).

### B.2 Repair and task specificity

**Table B.2:** Repair for every case with a gap of at least 5% (layers 8–20; variable copying has none). Share of the gap closed (%) by one and two same-task directions (top principal / answer-separating), by two generic-text and two random directions, and the number of directions that close half the gap. "–": not measured.

| Task | Layer | Gap | 1 direction | 2 directions | Generic (2) | Random (2) | For half |
|---|---|---|---|---|---|---|---|
| Running sum | 12 | 6 | −2 / 9 | −4 / 18 | 0 | 1 | >32 / >32 |
| | 14 | 6 | −1 / 14 | −5 / 22 | −1 | 2 | >32 / >32 |
| | 16 | 5 | −4 / 36 | 6 / 46 | – | −1 | 32 / 4 |
| | 18 | 13 | 23 / 44 | 25 / **58** | 0 | 1 | 8 / **2** |
| | 20 | 14 | 19 / 14 | 23 / 43 | – | 0 | 8 / 4 |
| Object tracking | 8 | 5 | −16 / −8 | −26 / 0 | – | 0 | >32 / >32 |
| | 12 | 26 | **52** / 38 | 53 / 39 | 1 | 0 | **1** / 8 |
| | 18 | 14 | −2 / 26 | 18 / 36 | 2 | 1 | 16 / 4 |
| | 20 | 10 | −32 / 29 | −6 / 47 | – | −2 | 16 / 4 |
| Names (IOI) | 14 | 10 | 1 / 2 | 1 / 2 | 1 | 0 | >32 / >32 |
| | 16 | 11 | −1 / −3 | −13 / −6 | – | 0 | >32 / >32 |
| | 18 | 24 | 6 / 5 | 17 / 9 | 0 | 0 | 32 / 32 |
| | 20 | 36 | 10 / 9 | 19 / 15 | – | 0 | 16 / 16 |

Only two cases are concentrated, with one or two directions closing half the gap: object tracking at layer 12 and the running sum at layer 18. The other gaps need 4 to more than 32 directions. Small gaps give noisy, sometimes negative, shares. Table B.3 compares sources for the two concentrated cases.

**Table B.3:** Share of the gap closed (%) by two directions from each source.

| Task (gap) | Source | Share closed |
|---|---|---|
| Object tracking, layer 12 (26%) | same task: top principal / answer-separating | 53 / 39 |
| | other task: names / running sum | 2 / −1 |
| | generic text / random | 1 / 0 |
| Running sum, layer 18 (13%) | same task: top principal / answer-separating | 25 / 58 |
| | other task: object tracking / names | 1 / 0 |
| | generic text / random | 0 / 1 |

### B.3 SAE width

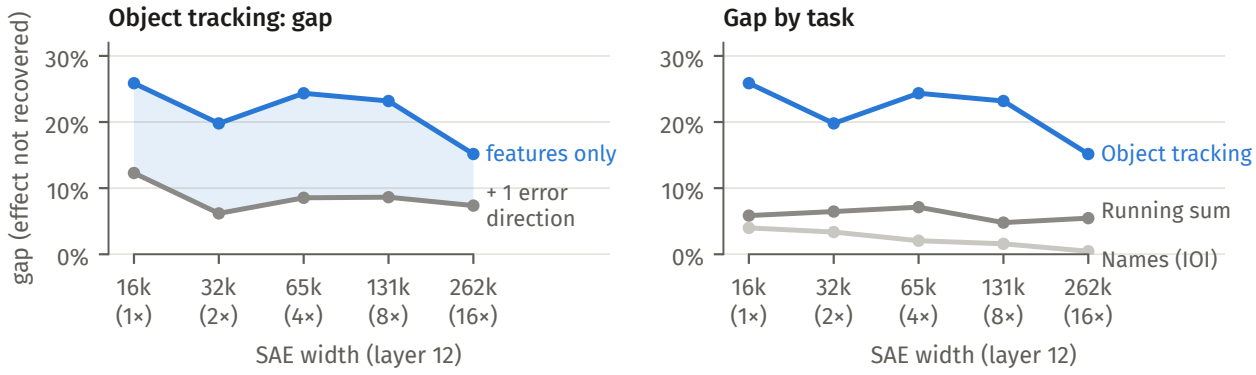



**Figure B.2:** Wider SAEs at layer 12. Left: the object-tracking gap with the features only and with one error direction added; the shaded band is what that direction closes. Right: the gap for three tasks.

**Table B.4:** Object tracking across SAE widths, layer 12 (%). "+1 error" and "+1 random" add one top principal or one random direction. "Active" is the mean number of features in use per token.

| Width | Features only [95% CI] | Gap | +1 error | +1 random | Active |
|---|---|---|---|---|---|
| 16k | 74 [73, 76] | 26 | 88 | 74 | 78 |
| 32k | 80 [79, 82] | 20 | 94 | 80 | 75 |
| 65k | 76 [74, 77] | 24 | 91 | 76 | 76 |
| 131k | 77 [75, 78] | 23 | 91 | 77 | 131 |
| 262k | 85 [84, 86] | 15 | 93 | 85 | 124 |

The 262k SAE keeps 59% of the 16k gap, just under the 60% we had set in advance for calling a gap persistent. The 131k and 262k SAEs also use more features per token, so part of the drop may come from sparsity rather than width. For the running sum and names tasks the gap stays at 0–7% at every width.

### B.4 Three measurement pitfalls

**1. Patching only the last token.** In early and middle layers, the information still sits on earlier tokens. Patching only the last token then recovers little, even with the whole activation (Table B.5). Patching from the first changed token recovers 100% at every task and layer [15].

**Table B.5:** Effect recovered (%) when the whole activation is patched at the last token only.

| Task | Layer 4 | Layer 8 | Layer 12 | Layer 16 | Layer 20 |
|---|---|---|---|---|---|
| Running sum | 0 | 0 | 3 | 58 | 94 |
| Object tracking | 1 | 4 | 47 | 77 | 85 |
| Variable copying | 0 | 0 | 0 | 5 | 55 |
| Names (IOI) | 0 | 0 | 0 | 60 | 81 |

The next two pitfalls concern *probes*: linear classifiers that read the answer from the last token's activation (Table B.6).

**2. The prompt can give the answer away.** A probe on the prompt's tokens alone scores 100% on variable copying and names. A high probe score there says nothing about the model's internal state [7].

**3. Readable is not the same as used.** For object tracking and names, probes read the answer far better from $\varepsilon$ than from $\hat{x}$ (84 vs 48%; 97 vs 20%). Yet patching that token recovers at most 47%: the model does not yet use the information at that position.

**Table B.6:** Probe accuracy at the last token, layer 12 (%), and the effect of patching that token.

| Task | Probe on $x$ | On $\varepsilon$ | On $\hat{x}$ | Prompt tokens only | Chance | Last-token patch |
|---|---|---|---|---|---|---|
| Running sum | 57 | 53 | 42 | 45 | 25 | 3 |
| Object tracking | 91 | 84 | 48 | 33 | 18 | 47 |
| Variable copying | 98 | 95 | 39 | 100 | 11 | 0 |
| Names (IOI) | 100 | 97 | 20 | 100 | 11 | 0 |

### B.5 Robustness and caveats

**Replication.** A separate, smaller run (160 training and 80 test pairs) agrees within one or two points. For object tracking at layer 12, the features recover 75% (main run: 74%) and one direction closes 51% of the gap (52%). The names gap at layer 18 is 24% in both runs.

**Caveats.**

- Object tracking is the hardest task: only 27% of generated pairs pass the filter. Its test pairs may over-represent easier cases.
- The names task's late-layer loss is spread over many directions. Part of what SAEs miss is therefore a general weakness, not a task-specific gap.

## C Design of the planned study

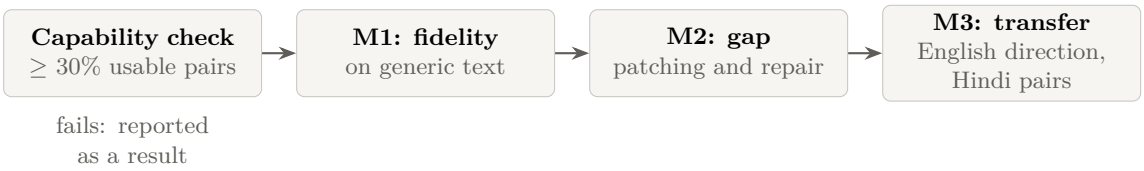



**Figure C.1:** Steps for each language. A language goes on to the next step only if the model can do the tasks in it.

**Table C.1:** Multilingual measures and criteria (95% intervals; otherwise inconclusive).

| ID | Measure | Supported if | Not supported if |
|---|---|---|---|
| M1 | fidelity on generic text, per language | Hindi ≥ 3 points below English | within 1 point of English |
| M2 | gap and specificity on the tasks | Hindi gap larger, non-overlapping intervals | Hindi gap not larger |
| M3 | English direction applied to Hindi pairs | closes ≥ half as much as Hindi's own direction | closes under a quarter as much |

**Languages.** English is the baseline. Hindi in Devanagari is the best-resourced Indian language and the most likely to work. Hindi in Latin script without English words and Hinglish (Latin script mixed with English words) together separate the effect of script from that of vocabulary. Tamil or Bengali are added if the model can do the tasks. French is a high-resource, non-Indian control. Q2 (binding) is run in English only.

**Templates.** The answer must be the last token. Arithmetic templates end with "=", as in English. Where Hindi puts the verb last, the template ends with a question followed by "answer:". Examples:

- Object tracking (Hindi): सिक्का डिब्बे B में है। चाबी डिब्बे E में है। चाबी डिब्बे C में चली जाती है। चाबी किस डिब्बे में है? डिब्बा → C
- Names (Hinglish): `Jab Rahul aur Priya market gaye, Priya ne ek book di. Book kisko mili? Answer:` → Rahul
- Variable copying is code-like and identical in every language. It serves as a language-neutral control.

**Data.** Fidelity per language uses generic text from IndicCorp v2 [3] and FineWeb2 [12].

**Risks and fallbacks.**

- Binding vectors may not be estimable in gemma-2-2b. Q2 then keeps only the shifting test.
- The model may not do the tasks in Tamil or Bengali. We then report this and keep Hindi and Hinglish.
- Many Hindi names split into several tokens. We use single-token names, or letters and digits as answers.
- Width results at one layer may not generalise. Q1 repeats them at a second layer if time allows.

# D Glossary

**Table D.1:** Terms used in this proposal.

| Term | Meaning |
|---|---|
| Token | a word or word piece; the unit the model reads |
| Layer | one of gemma-2-2b's 26 processing stages |
| Activation $x$ | the model's internal vector for one token at one layer (2304 numbers) |
| Sparse autoencoder (SAE) | a tool that rewrites an activation using a few interpretable features |
| Features $\hat{x}$ | the SAE's reconstruction of $x$ |
| SAE error $\varepsilon$ | $x - \hat{x}$: what the SAE does not explain ("dark matter", the leftovers) |
| Width | how many features an SAE can choose from (16k = 16,384, up to 262k) |
| Active features | the number of features in use for one token |
| Reconstruction fidelity | share of next-token performance kept when $\hat{x}$ replaces $x$ |
| Prompts A and B | two prompts that differ in one step, so the right answer differs |
| Activation patching | running the model on B while replacing part of its activation with A's at one layer |
| Effect recovered | how far patching moves the model's answer from B's towards A's, from 0 to 100% |
| Gap | the share of the effect the features miss: 100% minus the effect recovered by the features |
| Direction | a fixed vector in the activation space |
| Top principal directions | the directions in which a set of errors varies most |
| Answer-separating directions | the directions that best separate the average errors for different answers |
| Specificity score | share of the gap closed by same-task directions minus the largest share from any other source |
| Probe | a simple linear classifier trained to read a value, such as the box, from an activation |
| Binding | how a model links an entity to its attribute, such as an object to its box |
| IOI | "indirect object identification", the names task [14] |
| Generic text | ordinary text from the Pile |
| Hinglish | Hindi written in Latin script and mixed with English words |
| 95% CI | bootstrap confidence interval: the range the result would likely fall in on new test pairs |